

Lab 11 Answers

Yanqiao Chen

Department of Computer Science and Engineering
Southern University of Science and Technology
12412115@mail.sustech.edu.cn

May 18, 2026

1 Question 1

1.1 a)

We construct a verifier V for 3CNF formula: For a assignment x and a 3CNF formula ϕ , V checks if x satisfies ϕ . If it does, V accepts; otherwise, it rejects. Since the length of x is polynomial in the length of ϕ , and V runs in polynomial time, we have that 3-SAT is in NP.

1.2 b)

Given a k vertex cover, we can verify if it is a valid vertex cover by checking if every edge in the graph has at least one endpoint in the vertex cover. Since there are at most $O(|E|)$ edges in the graph, and checking each edge takes constant time, the total verification time is polynomial in the size of the graph. Therefore, vertex cover is in NP.

1.3 c)

Given a TSP route, we can verify if it is a valid TSP route by checking if it visits each city exactly once and returns to the starting city. We can also calculate the total distance of the route and check if it is less than or equal to a given threshold. Since there are at most $O(n)$ cities, and checking each city and calculating the distance takes polynomial time, the total verification time is polynomial in the size of the input. Therefore, TSP is in NP.

1.4 d)

Given a max-cut solution for the graph, we can verify if it is a valid max-cut solution by checking if the cut divides the vertices into two disjoint sets and if the number of edges crossing the cut is greater than or equal to a given threshold. Since there are at most $O(|V|)$ vertices and $O(|E|)$ edges in the graph, and checking each vertex and edge takes polynomial time, the total verification time is polynomial in the size of the graph. Therefore, max-cut is in NP.

1.5 e)

Given a 3-coloring of a graph, we can verify if it is a valid 3-coloring by checking if each vertex is colored with one of three colors and if no two adjacent vertices share the same color. Since there are at most $O(|V|)$ vertices and $O(|E|)$ edges in the graph, and checking each vertex and edge takes polynomial time, the total verification time is polynomial in the size of the graph. Therefore, 3-coloring is in NP.

2 Question 2

We now show that **NP** is closed under union, concatenation, and star operations.

- **Union:** Let L_1 and L_2 be two languages in NP. Then there exist two polynomial-time verifiers V_1 and V_2 for L_1 and L_2 , respectively. We can construct a new verifier V for the union language $L = L_1 \cup L_2$ as follows: Given an input string x and a certificate c , V runs both V_1 and V_2 on x and c . If either verifier accepts, then V accepts; otherwise, it rejects. Since both verifiers run in polynomial time, the new verifier also runs in polynomial time. Therefore, **NP** is closed under union.
- **Concatenation:** Let L_1 and L_2 be two languages in NP. Then there exist two polynomial-time verifiers V_1 and V_2 for L_1 and L_2 , respectively. We can construct a new verifier V for the concatenation language $L = L_1 L_2$ as follows: Given an input string x and a certificate (i, c_1, c_2) (including the split position), the verifier runs V_1 on c_1 in the first part of x and V_2 on c_2 in the second part, both in polynomial time. Therefore, **NP** is closed under concatenation.
- **Star:** Let L be a language in NP. The certificate contains two parts: solution and the splitting notation. The verifier V for the star language L^* works as follows: Given an input string x , V uses the splitting notation to split x into substrings. Then, for each substring, V runs the verifier for L on it. The certificate will be $((i_1, i_2, \dots, i_{k-1}), (c_1, c_2, \dots, c_k))$. If all substrings are accepted by the verifier for L , then V accepts; otherwise, it rejects. Since there are at most $O(|x|)$ substrings and each verifier runs in polynomial time, the new verifier also runs in polynomial time. Therefore, **NP** is closed under star.

3 Question 3

- a. Yes. All language in **P** can be verified in polynomial time. For the string x and a certificate c , we construct the following verifier V : V runs the polynomial-time algorithm for the language on input x . If it accepts, then V accepts; otherwise, it rejects. Since the algorithm runs in polynomial time, the verifier also runs in polynomial time. Therefore, **P** $\subseteq$ **NP**. (This means that we simply ignore the certificate and run the algorithm for the language.)
- b. True. Let M' simulate the behavior of M on input x , if M accepts x then M' reject, and if M rejects x then M' accepts. Since M runs in polynomial

time, M' also runs in polynomial time. Therefore, if a language is in $\mathbf{P}$, then its complement is also in $\mathbf{P}$.

- c. False. $\overline{SAT}$ is $co - NP$ -complete. If $\overline{SAT}$ is in $\mathbf{P}$, then $co - NP$ is a subset of $\mathbf{P}$. Since $\mathbf{P}$ is closed under complement, we have that NP is also a subset of $\mathbf{P}$. Therefore, $\mathbf{P} = \mathbf{NP} = \mathbf{coNP}$. However, it is widely believed that $\mathbf{P} \neq NP$, so we conclude that $\overline{SAT}$ is not in $\mathbf{P}$.
- d. True. From b we know that if L is in $\mathbf{P}$, then $\overline{L}$ is also in $\mathbf{P}$. Then $\mathbf{P} = \mathbf{coP}$. Therefore, since $\mathbf{P} \subseteq \mathbf{NP}$ and $\mathbf{coP} \subseteq \mathbf{coNP}$, thus $\mathbf{P} \subseteq \mathbf{NP} \cap \mathbf{coNP}$.
- e. True. If $\mathbf{P} = \mathbf{NP}$, then $\mathbf{coP} = \mathbf{coNP}$, and thus $\mathbf{P} = \mathbf{NP} = \mathbf{coP} = \mathbf{coNP}$. Therefore, $\mathbf{NP} = \mathbf{coNP}$.

4 Question 4

2-Color is in $\mathbf{NP}$ for we can construct a verifier that checks if a given coloring of the graph is valid. However, 2-Color is also in $\mathbf{P}$ because we can determine if a graph is bipartite (which is equivalent to being 2-colorable) using a two-coloring algorithm that runs in linear time. Therefore, 2-Color is in both $\mathbf{P}$ and $\mathbf{NP}$. We describe the algorithm for 2-coloring a graph as follows:

Algorithm 1 2-Coloring Algorithm

```

1: procedure TwoColor( $G$ )
2:   Initialize a color array  $color$  of size  $|V|$  with all values set to -1 (indicating
   uncolored).
3:   for each vertex  $v$  in  $V$  do
4:     if  $color[v] = -1$  then                                ▷ If the vertex is uncolored
5:        $color[v] \leftarrow 0$                                   ▷ Start coloring with color 0
6:       Create a queue  $Q$  and enqueue  $v$ 
7:       while  $Q$  is not empty do
8:          $u \leftarrow Q.dequeue()$ 
9:         for each neighbor  $w$  of  $u$  do
10:          if  $color[w] = -1$  then                            ▷ If the neighbor is uncolored
11:             $color[w] \leftarrow 1 - color[u]$                 ▷ Color with opposite color
12:            Enqueue  $w$ 
13:          else if  $color[w] = color[u]$  then                  ▷ If the neighbor has the
            same color
14:            return False                                    ▷ The graph is not 2-colorable
15:          end if
16:        end for
17:      end while
18:    end if
19:  end for
20:  return True                                              ▷ The graph is 2-colorable
21: end procedure

```

5 Question 5

We can verify if a given assignment satisfies a 2-CNF formula in polynomial time by checking each clause in the formula. Each clause has at most two literals, so we can check if at least one of the literals is true under the given assignment. If all clauses are satisfied, then the formula is satisfied; otherwise, it is not. Therefore, 2-SAT is in **NP**.

For any clauses in the 2-CNF formula, $(x_i \vee x_j) = (\neg x_i \rightarrow x_j) \wedge (\neg x_j \rightarrow x_i)$, this can be transformed into an implication graph where each variable and its negation are represented as vertices, and the implications are represented as directed edges. To determine if the 2-CNF formula is satisfiable, we can check if a variable and its negation are in the same strongly connected component in the implication graph. If any variable and its negation belong to the same SCC, then the formula is unsatisfiable; otherwise, it is satisfiable. This can be done using depth-first search (DFS) or breadth-first search (BFS) in linear time with respect to the number of variables and clauses. Therefore, 2-SAT can be solved in polynomial time.

The algorithm for solving 2-SAT is as follows:

Algorithm 2 2-SAT Algorithm

```
1: procedure TwoSAT( $\phi$ )
2:   Construct the implication graph  $G$  from the 2-CNF formula  $\phi$ 
3:   Perform a depth-first search (DFS) on  $G$  to find strongly connected
      components (SCCs)
4:   for each variable  $x_i$  do
5:     if  $x_i$  and  $\neg x_i$  are in the same SCC then
6:       return False                                ▷ The formula is unsatisfiable
7:     end if
8:   end for
9:   return True                                       ▷ The formula is satisfiable
10: end procedure
```
